Prelectura obligatoria: Fundamentos de R para lingüística de corpus

Módulo de Análisis Exploratorio de Datos — Doctorado en Lingüística

> Basado en Gries, S. Th. (2009). *Quantitative Corpus Linguistics with R*, cap. 3. Londres: Routledge.

Antes de empezar

Por qué esta lectura es obligatoria

Las cinco sesiones del módulo están dedicadas al análisis exploratorio, no a la sintaxis de R. Eso solo funciona si llegas a la primera sesión capaz de crear un vector, filtrar un data frame y escribir un bucle simple. Este documento cubre exactamente ese mínimo.

No necesitas memorizar nada. Necesitas haber **ejecutado** el código, haber visto los mensajes de error y haber resuelto las actividades. La diferencia entre leer código y ejecutarlo es la misma que hay entre leer una gramática y hablar la lengua.

Cómo trabajar este documento

1. Abre RStudio con un script vacío junto a este texto.
2. Escribe el código a mano —no copies y pegues. Los errores de tipeo enseñan.
3. Resuelve las actividades al final de cada sección antes de avanzar.
4. Anota las dudas que queden: la primera sesión abre con veinte minutos para resolverlas.

Instalación

Necesitas dos programas, en este orden:

- **R** — el lenguaje: https://cran.r-project.org
- **RStudio Desktop** — el entorno de trabajo: https://posit.co/download/rstudio-desktop/

Verifica la instalación ejecutando esto en la consola:

```
R.version.string
#> [1] "R version 4.4.1 (2024-06-14)"
```

Si ves algo similar (la versión puede diferir), estás listo.

Convención tipográfica

En los bloques de código, las líneas que empiezan con `#>` muestran el **output** que R devuelve. Las que empiezan con `#` son comentarios explicativos. No escribas ninguna de las dos: son para leer.

3.1 Nociones centrales

3.1.1 R como calculadora

R evalúa expresiones. Escribe una y presiona Enter:

```
2 + 3
#> [1] 5

(4 + 6) / 2
#> [1] 5

2^10
#> [1] 1024
```

El `[1]` que precede al resultado no es parte del valor: es el **índice** del primer elemento mostrado en esa línea. Cobrará sentido cuando trabajemos con vectores largos.

3.1.2 Asignación: guardar resultados

Para conservar un resultado hay que asignarlo a un nombre. El operador de asignación es `<-`:

```
frecuencia <- 42
frecuencia
#> [1] 42
```

R también acepta `=`, pero la convención en la comunidad es `<-` y así lo verás en toda la bibliografía. En RStudio, el atajo `Alt + -` lo escribe automáticamente.

Los nombres son **case-sensitive** y no admiten espacios:

```
Frecuencia <- 100
frecuencia   # sigue siendo 42
#> [1] 42
Frecuencia
#> [1] 100
```

Esta es una fuente clásica de errores. `corpus` y `Corpus` son dos objetos distintos.

Reglas para nombrar objetos

- Pueden contener letras, números, puntos (`.`) y guiones bajos (`_`)
- No pueden empezar con número ni con guion bajo
- No uses nombres de funciones existentes (`mean`, `sum`, `data`): R lo permite, pero luego te sorprenderá
- Prefiere `snake_case`: `frecuencia_relativa`, no `FrecuenciaRelativa`

```
mi_corpus <- 10       # válido
corpus.2024 <- 20     # válido
2corpus <- 30         # ERROR: empieza con número
mi corpus <- 40       # ERROR: contiene espacio
```

3.1.3 Tipos de datos básicos (data types)

R distingue varios tipos. Los cuatro que importan ahora:

Tipo	Qué contiene	Ejemplo
`numeric`	Números (con o sin decimales)	`3.14`, `42`
`character`	Texto, entre comillas	`"sustantivo"`
`logical`	Valores de verdad	`TRUE`, `FALSE`
`integer`	Enteros estrictos	`5L`

Para averiguar el tipo de un objeto se usa `class()`:

```
palabra <- "gato"
class(palabra)
#> [1] "character"

frecuencia <- 42
class(frecuencia)
#> [1] "numeric"

es_sustantivo <- TRUE
class(es_sustantivo)
#> [1] "logical"
```

Cuidado con las comillas. Sin ellas, R busca un objeto con ese nombre:

```
class(gato)
#> Error: object 'gato' not found
```

El texto siempre va entre comillas —dobles o simples, R no distingue, pero conviene ser consistente.

`TRUE` y `FALSE` van en mayúsculas y **sin** comillas. `"TRUE"` entre comillas es texto, no un valor lógico.

3.1.4 Funciones y arguments

Una función recibe **arguments** y devuelve un resultado. La sintaxis general es `nombre_funcion(argumento1, argumento2, ...)`:

```
sqrt(16)
#> [1] 4

round(3.14159, digits = 2)
#> [1] 3.14

nchar("lingüística")
#> [1] 11
```

Los arguments pueden pasarse por **posición** o por **nombre**:

```
round(3.14159, 2)              # por posición
#> [1] 3.14

round(x = 3.14159, digits = 2) # por nombre
#> [1] 3.14

round(digits = 2, x = 3.14159) # por nombre, orden alterado: funciona igual
#> [1] 3.14
```

Nombrar los arguments hace el código más legible y evita errores. Es especialmente recomendable cuando la función tiene más de dos.

Muchos arguments tienen **valores por defecto**. Si no los especificas, R usa el default:

```
round(3.14159)     # digits tiene default = 0
#> [1] 3
```

Consultar la ayuda

Toda función documentada tiene su página de ayuda. Se accede con `?`:

```
?round
?nchar
```

La página tiene siempre la misma estructura: *Description*, *Usage*, *Arguments*, *Details*, *Value*, *Examples*. La sección **Arguments** es la que consultarás más. Los **Examples** del final son ejecutables: cópialos y pruébalos.

Si no recuerdas el nombre exacto de una función, `??` busca en toda la documentación:

```
??"standard deviation"
```

3.1.5 Funciones propias

Cuando repites la misma operación, conviene encapsularla. La sintaxis es:

```
nombre <- function(argumentos) {
  cuerpo
  resultado
}
```

Un ejemplo con sentido lingüístico. La **frecuencia relativa normalizada** por cada mil palabras se calcula como `(frecuencia_absoluta / tamaño_corpus) * 1000`. Escribamos la función:

```
frecuencia_normalizada <- function(f_absoluta, tam_corpus, base = 1000) {
  (f_absoluta / tam_corpus) * base
}
```

Ahora la usamos:

```
# 340 ocurrencias en un corpus de 85.000 palabras
frecuencia_normalizada(340, 85000)
#> [1] 4

# Normalizando por millón de palabras
frecuencia_normalizada(340, 85000, base = 1000000)
#> [1] 4000
```

Tres observaciones sobre esta función:

1. El argument `base` tiene un **valor por defecto** (`1000`), de modo que puede omitirse.
2. La función devuelve el valor de la **última expresión evaluada**. No hace falta `return()`, aunque es válido escribirlo.
3. Los objetos creados dentro de la función no existen fuera de ella. Esto se llama **scope** y evita que las funciones interfieran entre sí.

3.1.6 El working directory

R siempre trabaja "situado" en un directorio. Para saber cuál:

```
getwd()
#> [1] "/home/usuario"
```

Para cambiarlo:

```
setwd("/home/usuario/proyecto_corpus")
```

Recomendación fuerte: en lugar de usar `setwd()`, crea un **RStudio Project** (`File > New Project`). El working directory queda anclado a la carpeta del proyecto, y las rutas relativas funcionan en cualquier computador. Esto es la base de la reproducibilidad, tema de la Semana 5.

Actividades 3.1

A1.1 Crea tres objetos: uno `character` con tu apellido, uno `numeric` con el año de inicio de tu doctorado y uno `logical` que indique si tu investigación usa corpus. Verifica el `class` de cada uno.

A1.2 Consulta la ayuda de `toupper()` y `tolower()`. Aplica cada una al objeto con tu apellido. ¿Qué devuelven?

A1.3 La función `nchar()` cuenta caracteres. Calcula cuántos caracteres tiene la palabra `"paralelepípedo"`. Luego prueba con `"año"`. ¿El resultado es el esperado? (Pista: la codificación de caracteres no ASCII puede sorprender según el sistema.)

A1.4 Escribe una función llamada `type_token_ratio()` que reciba dos argumentos —número de types y número de tokens— y devuelva el TTR, es decir, `types / tokens`. Pruébala con un texto de 450 types y 1200 tokens.

A1.5 Modifica la función anterior para que acepte un tercer argument `porcentaje`, con default `FALSE`. Si es `TRUE`, el resultado debe multiplicarse por 100. (Pista: necesitarás `if`, que se explica en 3.6; puedes volver a esta actividad después de leer esa sección, o intentarlo ahora.)

A1.6 Ejecuta `?mean` y localiza el argument `na.rm`. ¿Cuál es su valor por defecto? ¿Para qué sirve? Anota la respuesta: será relevante cuando trabajemos con datos incompletos.

3.2 Vectores

3.2.1 Qué es un vector

Un **vector** es una secuencia ordenada de elementos **del mismo tipo**. Es la estructura de datos más básica de R y, de hecho, los valores individuales que vimos en 3.1 son vectores de longitud 1.

Se crean con la función `c()` (de *combine*):

```
frecuencias <- c(45, 23, 78, 12, 56)
frecuencias
#> [1] 45 23 78 12 56

palabras <- c("el", "de", "que", "y", "la")
palabras
#> [1] "el"  "de"  "que" "y"   "la"
```

La función `length()` devuelve el número de elementos:

```r
length(frecuencias)
#> [1] 5
```

3.2.2 Coerción: el mismo tipo, siempre

Si mezclas tipos, R no protesta: **convierte todo al tipo más general**. Esto se llama *coercion* y es una fuente frecuente de errores silenciosos:

```r
mezcla <- c(1, 2, "tres")
mezcla
#> [1] "1"    "2"    "tres"
class(mezcla)
#> [1] "character"
```

Los números se volvieron texto. La jerarquía de coerción es:

$$\texttt{logical} \rightarrow \texttt{integer} \rightarrow \texttt{numeric} \rightarrow \texttt{character}$$

Un caso menos obvio, pero muy útil:

```r
c(TRUE, FALSE, TRUE, TRUE)
sum(c(TRUE, FALSE, TRUE, TRUE))
#> [1] 3
```

`TRUE` se convierte en 1 y `FALSE` en 0, de modo que `sum()` sobre un vector lógico **cuenta cuántos son verdaderos**. Este idioma aparecerá constantemente.

3.2.3 Vectores con nombres

Los elementos pueden llevar nombre, lo que resulta natural para tablas de frecuencia:

```r
frecuencias <- c(el = 4521, de = 3890, que = 2103, y = 1987, la = 1854)
frecuencias
#>   el   de  que    y   la
#> 4521 3890 2103 1987 1854
```

Los nombres se consultan y modifican con `names()`:

```r
names(frecuencias)
#> [1] "el"  "de"  "que" "y"   "la"
```

3.2.4 Generar vectores automáticamente

Tres funciones que ahorran escritura:

```r
# Secuencia de enteros consecutivos
1:10
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

```r
# Secuencia con incremento arbitrario
seq(from = 0, to = 100, by = 25)
#> [1]   0  25  50  75 100

# Repetición
rep("NOUN", times = 3)
#> [1] "NOUN" "NOUN" "NOUN"

rep(c("a", "b"), times = 2)
#> [1] "a" "b" "a" "b"

rep(c("a", "b"), each = 2)
#> [1] "a" "a" "b" "b"
```

La diferencia entre `times` y `each` es sutil y conviene fijarla: `times` repite el vector completo; `each` repite cada elemento.

3.2.5 Indexing: acceder a los elementos

El **indexing** (o *subsetting*) se hace con corchetes. R indexa desde **1**, no desde 0:

```r
palabras <- c("el", "de", "que", "y", "la")

palabras[1]
#> [1] "el"

palabras[3]
#> [1] "que"
```

Se pueden pedir varios elementos pasando un vector de índices:

```r
palabras[c(1, 3, 5)]
#> [1] "el"  "que" "la"

palabras[2:4]
#> [1] "de"  "que" "y"
```

Un índice **negativo** excluye:

```r
palabras[-1]
#> [1] "de"  "que" "y"   "la"

palabras[-c(1, 2)]
#> [1] "que" "y"   "la"
```

Si el vector tiene nombres, se puede indexar por nombre:

```r
frecuencias["que"]
#>  que
#> 2103
```

3.2.6 Logical indexing: la técnica central

Esta es la forma de indexing más importante para el análisis de datos. Consiste en pasar un vector lógico: R devuelve los elementos donde hay `TRUE`.

```
frecuencias <- c(el = 4521, de = 3890, que = 2103, y = 1987, la = 1854)

# ¿Qué elementos superan las 2000 ocurrencias?
frecuencias > 2000
#>    el    de   que     y    la
#>  TRUE  TRUE  TRUE FALSE FALSE

# Devolver solo esos
frecuencias[frecuencias > 2000]
#>   el   de  que
#> 4521 3890 2103
```

Los operadores de comparación son:

Operador	Significado
`==`	igual a
`!=`	distinto de
`> <`	mayor / menor que
`>= <=`	mayor / menor o igual
`%in%`	pertenece a

El error más común de todos: usar `=` en lugar de `==` para comparar. `=` asigna; `==` compara.

Las condiciones se combinan con `&` (y), `|` (o) y `!` (negación):

```
frecuencias[frecuencias > 1900 & frecuencias < 4000]
#>   de  que    y
#> 3890 2103 1987
```

El operador `%in%` es especialmente útil para filtrar por una lista de valores:

```
palabras <- c("el", "de", "que", "y", "la")
funcionales <- c("el", "la", "los", "las")

palabras[palabras %in% funcionales]
#> [1] "el" "la"
```

3.2.7 Operaciones vectorizadas

R aplica las operaciones **elemento por elemento**, sin necesidad de bucles. Esto se llama *vectorization* y es una de las características que definen el lenguaje:

```r
frecuencias_abs <- c(340, 210, 95, 47)
tam_corpus <- 85000

# Una sola operación sobre los cuatro valores
frecuencias_norm <- (frecuencias_abs / tam_corpus) * 1000
frecuencias_norm
#> [1] 4.000000 2.470588 1.117647 0.552941
```

Compara esto con lo que exigiría un lenguaje sin vectorización: un bucle recorriendo cada elemento. En R, escribir ese bucle sería redundante y más lento.

Las funciones de resumen operan sobre el vector completo:

```r
sum(frecuencias_abs)
#> [1] 692
mean(frecuencias_abs)
#> [1] 173
max(frecuencias_abs)
#> [1] 340
range(frecuencias_abs)
#> [1]  47 340
```

3.2.8 Ordenar

Dos funciones que se confunden con facilidad:

- `sort()` devuelve el **vector ordenado**
- `order()` devuelve los **índices** que producirían ese orden

```r
frecuencias <- c(el = 4521, de = 3890, que = 2103, y = 1987, la = 1854)

sort(frecuencias)
#>   la    y  que   de   el
#> 1854 1987 2103 3890 4521

sort(frecuencias, decreasing = TRUE)
#>   el   de  que    y   la
#> 4521 3890 2103 1987 1854
```

`order()` parece menos útil, pero es imprescindible para ordenar data frames (sección 3.4):

```r
order(frecuencias)
#> [1] 5 4 3 2 1
```

Se lee así: "el elemento más pequeño está en la posición 5, el segundo más pequeño en la 4, ...". Y en efecto, `frecuencias[order(frecuencias)]` produce el mismo resultado que `sort(frecuencias)`.

3.2.9 Guardar y cargar vectores

Para escribir un vector a un archivo de texto:

```
palabras <- c("el", "de", "que", "y", "la")
writeLines(palabras, "palabras.txt")
```

Para leerlo de vuelta:

```
palabras_leidas <- readLines("palabras.txt")
palabras_leidas
#> [1] "el"  "de"  "que" "y"   "la"
```

`readLines()` es la función que usarás para cargar textos de corpus: devuelve un vector donde cada elemento es una línea del archivo.

3.2.10 Un gráfico rápido

Con un vector con nombres, `barplot()` produce un gráfico de frecuencias inmediato:

```
frecuencias <- c(el = 4521, de = 3890, que = 2103, y = 1987, la = 1854)

barplot(frecuencias,
        main = "Cinco palabras más frecuentes",
        ylab = "Frecuencia absoluta",
        col = "steelblue")
```

Este es el gráfico base de R. En el módulo trabajaremos también con `ggplot2`, más flexible, pero `barplot()` sirve perfectamente para una inspección rápida.

Actividades 3.2

A2.1 Crea un vector llamado `pos_freq` con las siguientes frecuencias de etiquetas POS: `NOUN = 1250`, `VERB = 890`, `ADJ = 445`, `ADV = 312`, `PRON = 678`. Verifica su `length` y sus `names`.

A2.2 Usando `pos_freq`, extrae: (a) el tercer elemento, (b) los elementos primero y último, (c) todos menos `ADV`, (d) el elemento llamado `"VERB"` usando indexing por nombre.

A2.3 Aplica logical indexing para obtener solo las etiquetas con frecuencia superior a 500. ¿Cuántas hay? Resuélvelo también con `sum()` sobre el vector lógico.

A2.4 Calcula la frecuencia relativa de cada etiqueta como porcentaje del total. Guarda el resultado en un vector `pos_pct` y redondéalo a dos decimales.

A2.5 Ordena `pos_freq` de mayor a menor. Luego obtén el nombre de la etiqueta más frecuente **sin mirarlo a ojo**, usando `names()` y `which.max()`.

A2.6 Genera un `barplot` de `pos_pct` con título y etiqueta del eje Y apropiados. Añade el argument `las = 2` y observa qué cambia.

A2.7 Un corpus tiene 45.000 tokens. Las frecuencias absolutas de cinco marcadores discursivos son `c(89, 156, 34, 201, 67)`. Calcula sus frecuencias normalizadas por millón de palabras usando una sola expresión vectorizada.

A2.8 Crea un vector de 20 números aleatorios entre 1 y 100 con `sample(1:100, 20)`. Extrae los que sean pares. (Pista: el operador módulo es `%%`; un número es par si `x %% 2 == 0`.)

3.3 Factores

3.3.1 Qué es un factor y por qué importa

Un **factor** es la estructura que R usa para representar **variables categóricas**. Internamente almacena enteros, pero muestra etiquetas —los **levels**.

La conexión con el módulo es directa: los factores son la implementación computacional de las escalas nominal y ordinal que revisamos en los fundamentos de muestreo. Un factor sin orden es una variable nominal; un factor con `ordered = TRUE` es ordinal.

```
pos <- factor(c("NOUN", "VERB", "ADJ", "NOUN", "VERB", "NOUN"))
pos
#> [1] NOUN VERB ADJ  NOUN VERB NOUN
#> Levels: ADJ NOUN VERB
```

Nota que `Levels` aparece por separado: son las categorías posibles, no los valores observados. Por defecto R las ordena alfabéticamente.

```
levels(pos)
#> [1] "ADJ"  "NOUN" "VERB"

nlevels(pos)
#> [1] 3

class(pos)
#> [1] "factor"
```

3.3.2 Contar: `table()`

La función más usada con factores es `table()`, que produce una tabla de frecuencias:

```
table(pos)
#> pos
```

```
#>  ADJ NOUN VERB
#>    1    3    2
```

`table()` también funciona con vectores `character`, pero con factores tiene una ventaja: **incluye los levels con frecuencia cero**, lo cual es importante para no perder categorías al filtrar.

```
pos_filtrado <- pos[pos != "ADJ"]
table(pos_filtrado)
#> pos_filtrado
#>  ADJ NOUN VERB
#>    0    3    2
```

El level `ADJ` sigue ahí con frecuencia 0. Si necesitas eliminarlo, usa `droplevels()`:

```
table(droplevels(pos_filtrado))
#>
#> NOUN VERB
#>    3    2
```

3.3.3 Reordenar levels

El orden alfabético por defecto rara vez es el que interesa. Se controla con el argument `levels`:

```
registro <- factor(c("formal", "coloquial", "formal", "neutro", "coloquial"))
levels(registro)
#> [1] "coloquial" "formal"    "neutro"

# Reordenar según un criterio propio
registro <- factor(registro, levels = c("coloquial", "neutro", "formal"))
levels(registro)
#> [1] "coloquial" "neutro"    "formal"
```

Esto no cambia los datos, solo el orden de las categorías —lo que afecta a las tablas y, sobre todo, al orden de las barras en los gráficos.

Una alternativa práctica es ordenar los levels **por frecuencia**, que suele ser lo que se quiere en un gráfico:

```
pos_ord <- factor(pos, levels = names(sort(table(pos), decreasing = TRUE)))
levels(pos_ord)
#> [1] "NOUN" "VERB" "ADJ"
```

3.3.4 Factores ordinales

Cuando las categorías tienen un orden intrínseco, hay que declararlo explícitamente:

```r
aceptabilidad <- factor(
  c("marginal", "aceptable", "inaceptable", "aceptable", "marginal"),
  levels = c("inaceptable", "marginal", "aceptable"),
  ordered = TRUE
)

aceptabilidad
#> [1] marginal    aceptable   inaceptable aceptable   marginal
#> Levels: inaceptable < marginal < aceptable
```

Los signos `<` en la salida indican que el orden es significativo. Ahora se pueden hacer comparaciones:

```r
aceptabilidad > "inaceptable"
#> [1]  TRUE  TRUE FALSE  TRUE  TRUE

median(aceptabilidad)
#> [1] marginal
#> Levels: inaceptable < marginal < aceptable
```

Y —esto es lo pedagógicamente valioso— R **impide** calcular la media:

```r
mean(aceptabilidad)
#> Warning: argument is not numeric or logical: returning NA
#> [1] NA
```

Que la mediana funcione y la media no es exactamente el argumento sobre qué estadísticos son legítimos según la escala de medición. Aquí el error lo produce el lenguaje, no el profesor.

3.3.5 La trampa de la conversión numérica

Este es el error más peligroso con factores, y conviene verlo antes de cometerlo:

```r
puntajes <- factor(c(10, 25, 30, 25, 10))
puntajes
#> [1] 10 25 30 25 10
#> Levels: 10 25 30

# Intento ingenuo de recuperar los números
as.numeric(puntajes)
#> [1] 1 2 3 2 1
```

R devolvió los **códigos internos de los levels**, no los valores. La conversión correcta pasa por `character`:

```r
as.numeric(as.character(puntajes))
#> [1] 10 25 30 25 10
```

Regla mnemotécnica: para sacar números de un factor, siempre `as.numeric(as.character(x))`.

3.3.6 Tablas de contingencia

Cruzar dos factores produce una tabla de contingencia, estructura central en lingüística de corpus:

```
pos      <- factor(c("NOUN", "VERB", "NOUN", "VERB", "NOUN", "VERB"))
registro <- factor(c("formal", "formal", "coloquial", "coloquial", "formal", "coloquial"))

table(pos, registro)
#>       registro
#> pos    coloquial formal
#>   NOUN         1      2
#>   VERB         2      1
```

Con `prop.table()` se obtienen proporciones:

```
prop.table(table(pos, registro))          # sobre el total
prop.table(table(pos, registro), margin = 1)  # por filas
prop.table(table(pos, registro), margin = 2)  # por columnas
```

El argument `margin` es una fuente habitual de confusión: `1` son filas, `2` son columnas.

Actividades 3.3

A3.1 Crea un factor `tiempo_verbal` a partir del vector `c("pretérito", "presente", "futuro", "presente", "pretérito", "presente")`. Consulta sus `levels` y verifica que el orden por defecto es alfabético.

A3.2 Reordena los levels de `tiempo_verbal` según la secuencia temporal lógica: pretérito, presente, futuro. Genera un `table()` antes y después y compara la salida.

A3.3 Genera un `barplot(table(tiempo_verbal))` con los levels reordenados. ¿Por qué el orden de los levels importa para la interpretación del gráfico?

A3.4 Construye un factor ordinal `nivel_mcer` con valores del conjunto A1, A2, B1, B2, C1, C2 para ocho aprendientes (invéntalos). Declara el orden correctamente. Calcula la mediana y verifica que `mean()` produce un warning.

A3.5 Usando el factor anterior, aplica logical indexing para obtener los aprendientes de nivel B2 o superior. (Pista: la comparación `>= "B2"` funciona si el factor es `ordered`.)

A3.6 Crea el factor `puntos <- factor(c(5, 8, 3, 8, 5))`. Aplica `as.numeric()` directamente y observa el resultado incorrecto. Luego aplica la conversión correcta y verifica con `sum()` que ahora obtienes 29.

A3.7 Con dos vectores de tu invención —uno de tipo de cláusula (`principal`/`subordinada`) y otro de modalidad (`oral`/`escrito`), ambos

de 12 elementos— construye una tabla de contingencia. Obtén después las proporciones por filas e interprétalas en una frase.

A3.8 Crea un factor con cinco categorías, filtra para eliminar una de ellas y comprueba con `table()` que el level persiste con frecuencia 0. Elimínalo con `droplevels()`.

3.4 Data frames

3.4.1 Qué es un data frame

Un **data frame** es una estructura bidimensional donde cada columna es un vector y todas las columnas tienen la misma longitud. Es el equivalente de una hoja de cálculo o de una tabla de base de datos, y es el formato en que trabajarás la mayor parte del tiempo.

La restricción clave: **dentro de una columna todos los valores son del mismo tipo, pero columnas distintas pueden tener tipos distintos**. Eso lo hace ideal para metadatos de corpus, donde conviven identificadores de texto, frecuencias numéricas y variables categóricas.

```
corpus_meta <- data.frame(
  texto     = c("T01", "T02", "T03", "T04", "T05"),
  genero    = c("narrativo", "académico", "narrativo", "periodístico", "académico"),
  tokens    = c(1250, 3400, 890, 2100, 4200),
  types     = c(520,  980,  410,  760,  1150),
  año       = c(1998, 2005, 2011, 2005, 2018)
)
```

```
corpus_meta
#>   texto       genero tokens types  año
#> 1   T01    narrativo   1250   520 1998
#> 2   T02    académico   3400   980 2005
#> 3   T03    narrativo    890   410 2011
#> 4   T04 periodístico   2100   760 2005
#> 5   T05    académico   4200  1150 2018
```

Nota importante: desde R 4.0, `data.frame()` ya **no** convierte texto en factor automáticamente. En versiones anteriores lo hacía, y buena parte de la bibliografía —incluido Gries— asume ese comportamiento. Si necesitas factores, conviértelos explícitamente o usa `stringsAsFactors = TRUE`.

3.4.2 Inspeccionar un data frame

Cinco funciones que conviene ejecutar siempre al abrir un dataset nuevo:

```r
dim(corpus_meta)      # dimensiones: filas y columnas
#> [1] 5 5

nrow(corpus_meta)     # número de filas
#> [1] 5

ncol(corpus_meta)     # número de columnas
#> [1] 5

names(corpus_meta)    # nombres de las columnas
#> [1] "texto"  "genero" "tokens" "types"  "año"

str(corpus_meta)      # estructura: tipos y primeros valores
#> 'data.frame':    5 obs. of  5 variables:
#>  $ texto : chr  "T01" "T02" "T03" "T04" ...
#>  $ genero: chr  "narrativo" "académico" "narrativo" "periodístico" ...
#>  $ tokens: num  1250 3400 890 2100 4200
#>  $ types : num  520 980 410 760 1150
#>  $ año   : num  1998 2005 2011 2005 2018
```

`str()` es la más informativa de todas: en una sola salida muestra dimensiones, nombres, tipos y una muestra de valores. Conviértelo en reflejo.

Otras dos, útiles con datasets grandes:

```r
head(corpus_meta, n = 3)   # primeras filas
tail(corpus_meta, n = 2)   # últimas filas
summary(corpus_meta)       # resumen estadístico por columna
```

3.4.3 Acceder a columnas

Tres notaciones equivalentes:

```r
corpus_meta$tokens          # con $
#> [1] 1250 3400  890 2100 4200

corpus_meta[["tokens"]]     # con doble corchete
corpus_meta[, "tokens"]     # con corchete simple, indicando columna
```

La notación `$` es la más legible y la que verás con más frecuencia. En RStudio, escribir `corpus_meta$` despliega la lista de columnas disponibles.

Una columna extraída es un vector, así que todo lo de la sección 3.2 aplica:

```r
mean(corpus_meta$tokens)
#> [1] 2368
```

3.4.4 Indexing bidimensional

Los data frames se indexan con `[fila, columna]`. Dejar un espacio vacío significa "todas":

```
corpus_meta[1, ]           # primera fila, todas las columnas
corpus_meta[, 3]           # todas las filas, tercera columna
corpus_meta[2, 3]          # fila 2, columna 3
#> [1] 3400

corpus_meta[1:3, c("texto", "tokens")]   # combinando rangos y nombres
#>   texto tokens
#> 1   T01   1250
#> 2   T02   3400
#> 3   T03    890
```

La coma es obligatoria. `corpus_meta[1]` sin coma devuelve la **primera columna**, no la primera fila —un error frecuente.

3.4.5 Filtrar filas

El logical indexing de 3.2 funciona igual, pero en la posición de las filas:

```
# Textos con más de 2000 tokens
corpus_meta[corpus_meta$tokens > 2000, ]
#>   texto       genero tokens types  año
#> 2   T02    académico   3400   980 2005
#> 4   T04 periodístico   2100   760 2005
#> 5   T05    académico   4200  1150 2018

# Textos académicos
corpus_meta[corpus_meta$genero == "académico", ]

# Condiciones combinadas
corpus_meta[corpus_meta$tokens > 1000 & corpus_meta$año > 2000, ]
```

La función `subset()` hace lo mismo con sintaxis más limpia:

```
subset(corpus_meta, tokens > 2000)
subset(corpus_meta, genero == "académico", select = c(texto, tokens))
```

Dentro de `subset()` no hace falta repetir `corpus_meta$` en cada condición.

3.4.6 Añadir y modificar columnas

Asignar a una columna que no existe la crea:

```
corpus_meta$ttr <- corpus_meta$types / corpus_meta$tokens
corpus_meta$ttr <- round(corpus_meta$ttr, 3)
```

```
corpus_meta
#>   texto       genero tokens types  año   ttr
#> 1   T01    narrativo   1250   520 1998 0.416
#> 2   T02    académico   3400   980 2005 0.288
#> 3   T03    narrativo    890   410 2011 0.461
#> 4   T04 periodístico   2100   760 2005 0.362
#> 5   T05    académico   4200  1150 2018 0.274
```

Para eliminar una columna, se le asigna `NULL`:

```
corpus_meta$año <- NULL
```

3.4.7 Ordenar

Aquí es donde `order()` (sección 3.2.8) muestra su utilidad:

```
# Ordenar por número de tokens, ascendente
corpus_meta[order(corpus_meta$tokens), ]

# Descendente
corpus_meta[order(-corpus_meta$tokens), ]

# Por dos criterios: primero género, luego tokens
corpus_meta[order(corpus_meta$genero, -corpus_meta$tokens), ]
```

3.4.8 Agregar por grupos

La función `aggregate()` calcula estadísticos por categoría. Es el equivalente al *split-apply-combine* que usaremos en la Semana 4:

```
aggregate(tokens ~ genero, data = corpus_meta, FUN = mean)
#>         genero tokens
#> 1    académico   3800
#> 2    narrativo   1070
#> 3 periodístico   2100
```

La notación `y ~ x` se lee "y en función de x" y reaparecerá en modelos estadísticos. Se pueden agregar varias variables:

```
aggregate(cbind(tokens, types) ~ genero, data = corpus_meta, FUN = mean)
```

3.4.9 Guardar y cargar

El formato estándar de intercambio es CSV:

```
write.csv(corpus_meta, "corpus_meta.csv", row.names = FALSE)

datos <- read.csv("corpus_meta.csv")
```

El argument `row.names = FALSE` evita que R añada una columna espuria con los números de fila. Omítelo y lo lamentarás al reabrir el archivo.

Para datos con caracteres no ASCII —tildes, eñes— conviene ser explícito con la codificación:

```
write.csv(corpus_meta, "corpus_meta.csv", row.names = FALSE, fileEncoding = "UTF-8")
datos <- read.csv("corpus_meta.csv", encoding = "UTF-8")
```

Si el separador es tabulación o punto y coma (frecuente en archivos generados en configuración regional española), usa `read.delim()` o especifica `sep`:

```
datos <- read.csv("archivo.csv", sep = ";", dec = ",")
```

3.4.10 Combinar data frames

Para **apilar** filas de dos data frames con las mismas columnas:

```
mas_textos <- data.frame(
  texto  = c("T06", "T07"),
  genero = c("narrativo", "académico"),
  tokens = c(1800, 2900),
  types  = c(640, 890),
  ttr    = c(0.356, 0.307)
)

corpus_completo <- rbind(corpus_meta, mas_textos)
nrow(corpus_completo)
#> [1] 7
```

Para **unir** dos tablas por una columna común, `merge()` es el equivalente de un JOIN:

```
anotaciones <- data.frame(
  texto    = c("T01", "T02", "T03"),
  anotador = c("AB", "CD", "AB")
)

merge(corpus_meta, anotaciones, by = "texto")
#>   texto    genero tokens types   ttr anotador
#> 1   T01 narrativo   1250   520 0.416       AB
#> 2   T02 académico   3400   980 0.288       CD
#> 3   T03 narrativo    890   410 0.461       AB
```

Por defecto `merge()` conserva solo las filas presentes en ambas tablas (*inner join*). Para conservar todas las del primero, `all.x = TRUE`.

Actividades 3.4

A4.1 Crea un data frame `hablantes` con seis filas y estas columnas: `id` (H01–H06), `edad` (numérica), `sexo` (`"F"`/`"M"`), `ciudad` (tres ciudades distintas) y `sujeto_nulo` (proporción entre 0 y 1). Inspecciónalo con `str()` y `summary()`.

A4.2 Extrae la columna `edad` con las tres notaciones (`$`, doble corchete, corchete simple). Verifica con `identical()` que las tres devuelven lo mismo.

A4.3 Filtra los hablantes menores de 40 años. Hazlo dos veces: con logical indexing y con `subset()`.

A4.4 Añade una columna `grupo_etario` que valga `"joven"` si la edad es menor a 40 y `"mayor"` en caso contrario. (Pista: `ifelse(condición, valor_si, valor_no)`.)

A4.5 Convierte `ciudad` y `sexo` en factores. Genera la tabla de contingencia entre ambos.

A4.6 Calcula la media de `sujeto_nulo` por ciudad con `aggregate()`. Luego hazlo por ciudad y sexo simultáneamente (pista: `sujeto_nulo ~ ciudad + sexo`).

A4.7 Ordena el data frame por `sujeto_nulo` de mayor a menor y quédate con las tres primeras filas.

A4.8 Guarda el data frame como CSV con codificación UTF-8. Ciérralo, vuelve a cargarlo y verifica con `str()` que los tipos se conservaron. ¿Qué pasó con los factores?

A4.9 Crea un segundo data frame con los años de escolaridad de cuatro de los seis hablantes. Únelo al primero con `merge()`. ¿Cuántas filas tiene el resultado? Repite con `all.x = TRUE` y explica la diferencia.

3.5 Listas

3.5.1 Qué es una lista

Una **lista** es un contenedor que puede alojar objetos de **cualquier tipo y de cualquier longitud**, incluidas otras listas. Es la estructura más flexible de R.

Si un vector es una fila de casilleros idénticos, una lista es un archivador donde cada cajón puede contener algo distinto: un número, un vector de texto, un data frame completo.

```
analisis <- list(
  nombre_corpus = "Corpus Oral de Santiago",
  n_textos      = 45,
  generos       = c("conversación", "entrevista", "narración"),
```

```
  frecuencias   = c(el = 4521, de = 3890, que = 2103),
  metadatos     = data.frame(id = 1:3, tokens = c(1200, 3400, 890))
)

length(analisis)
#> [1] 5
```

Cinco elementos de cinco tipos distintos, en un solo objeto.

3.5.2 Acceder a los elementos: `[` frente a `[[`

Esta distinción es la principal fuente de confusión con listas, y conviene fijarla desde el principio:

- `[ ]` devuelve una **sublista**
- `[[ ]]` devuelve el **contenido** del elemento

```
analisis["n_textos"]
#> $n_textos
#> [1] 45

class(analisis["n_textos"])
#> [1] "list"

analisis[["n_textos"]]
#> [1] 45

class(analisis[["n_textos"]])
#> [1] "numeric"
```

Con la notación `$` se obtiene lo mismo que con `[[ ]]`:

```
analisis$n_textos
#> [1] 45
```

Una analogía útil: si la lista es un tren de vagones, `[ ]` devuelve un vagón (que sigue siendo parte del tren), y `[[ ]]` devuelve el contenido del vagón.

Como los elementos son objetos completos, se pueden indexar en cadena:

```
analisis$generos[2]
#> [1] "entrevista"

analisis$frecuencias["que"]
#>  que
#> 2103

analisis$metadatos$tokens
#> [1] 1200 3400  890
```

3.5.3 Modificar una lista

```
# Añadir un elemento
analisis$fecha <- "2024-03-15"

# Modificar uno existente
analisis$n_textos <- 50

# Eliminar
analisis$fecha <- NULL

names(analisis)
#> [1] "nombre_corpus" "n_textos"      "generos"       "frecuencias"   "metadatos"
```

3.5.4 Por qué importan las listas

Dos razones prácticas.

Primera: las funciones estadísticas devuelven listas. Casi todos los resultados de tests en R son listas, y saber navegarlas te da acceso a componentes que la salida impresa no muestra:

```
x <- c(12, 15, 9, 22, 18, 14, 11)
y <- c(19, 24, 17, 28, 25, 21, 20)

resultado <- t.test(x, y)
class(resultado)
#> [1] "htest"

names(resultado)
#> [1] "statistic" "parameter" "p.value" "conf.int" "estimate" ...

resultado$p.value
#> [1] 0.001432

resultado$conf.int
#> [1] -10.628223  -3.371777
```

Extraer `$p.value` directamente es lo que permite automatizar el reporte de resultados sin copiarlos a mano.

Segunda: agrupan resultados heterogéneos. Cuando una función propia debe devolver más de una cosa, la lista es la solución:

```
resumir_texto <- function(tokens, types, genero) {
  list(
    ttr        = round(types / tokens, 3),
    tokens     = tokens,
    genero     = genero,
```

```
    evaluacion = if (types / tokens > 0.4) "alta diversidad" else "baja diversidad"
  )
}

r <- resumir_texto(1250, 520, "narrativo")
r$ttr
#> [1] 0.416
r$evaluacion
#> [1] "alta diversidad"
```

3.5.5 Aplicar funciones a listas: la familia `apply`

Las funciones `lapply()` y `sapply()` aplican una función a cada elemento de una lista o vector, evitando escribir bucles.

```
textos <- list(
  t1 = c("el", "gato", "come", "el", "pescado"),
  t2 = c("la", "casa", "es", "grande"),
  t3 = c("un", "perro", "corre", "por", "el", "parque", "verde")
)

# lapply devuelve una lista
lapply(textos, length)
#> $t1
#> [1] 5
#> $t2
#> [1] 4
#> $t3
#> [1] 7

# sapply simplifica a vector cuando puede
sapply(textos, length)
#> t1 t2 t3
#>  5  4  7
```

La diferencia es únicamente el formato de salida: `sapply()` es `lapply()` más un intento de simplificación. Para trabajo interactivo, `sapply()` es más cómodo; para código robusto, `lapply()` es más predecible.

Un ejemplo con más contenido lingüístico —el TTR de cada texto:

```
sapply(textos, function(x) round(length(unique(x)) / length(x), 3))
#>    t1    t2    t3
#> 0.800 1.000 1.000
```

Aquí `function(x) ...` es una **función anónima**: se define sobre la marcha, sin nombre, porque solo se usa una vez.

Actividades 3.5

A5.1 Crea una lista `mi_estudio` con al menos cuatro elementos de tipos distintos: un `character` con el título, un `numeric` con el número de participantes, un vector con los nombres de las variables medidas y un data frame pequeño con datos.

A5.2 Accede al tercer elemento con `[ ]` y con `[[ ]]`. Verifica el `class` de cada resultado y explica en una frase la diferencia.

A5.3 Extrae el segundo valor del vector de variables usando indexing encadenado en una sola expresión.

A5.4 Añade a la lista un elemento `fecha` con la fecha de hoy. Luego elimínalo. Comprueba con `names()` en cada paso.

A5.5 Ejecuta un `t.test()` entre dos vectores numéricos que inventes. Extrae de la lista resultante: el p-value, el estadístico t y los grados de libertad.

A5.6 Crea una lista con cuatro vectores de palabras de distinta longitud. Usa `sapply()` para obtener el número de tokens de cada uno y luego el número de types.

A5.7 Escribe una función `describir_vector()` que reciba un vector numérico y devuelva una lista con su media, mediana, desviación estándar y número de elementos. Aplícala a `c(23, 45, 12, 67, 34, 89, 21)`.

A5.8 Usando la lista de textos de la sección 3.5.5, calcula con `sapply()` la longitud media de palabra de cada texto. (Pista: `mean(nchar(x))`.)

3.6 Funciones elementales de programación

3.6.1 Condicionales: `if` y `else`

Un condicional ejecuta código solo si se cumple una condición:

```
frecuencia <- 150

if (frecuencia > 100) {
  print("Palabra frecuente")
}
#> [1] "Palabra frecuente"
```

Con alternativa:

```
frecuencia <- 45

if (frecuencia > 100) {
  print("Palabra frecuente")
} else {
```

```
  print("Palabra poco frecuente")
}
#> [1] "Palabra poco frecuente"
```

Con varias condiciones encadenadas:

```
clasificar_frecuencia <- function(f) {
  if (f > 1000) {
    "muy alta"
  } else if (f > 100) {
    "alta"
  } else if (f > 10) {
    "media"
  } else {
    "baja"
  }
}

clasificar_frecuencia(2500)
#> [1] "muy alta"
clasificar_frecuencia(45)
#> [1] "media"
```

Limitación importante: `if` evalúa **un solo valor**. No funciona sobre vectores:

```
frecuencias <- c(45, 150, 890)
if (frecuencias > 100) print("alta")
#> Error in if (frecuencias > 100) : the condition has length > 1
```

3.6.2 `ifelse()`: el condicional vectorizado

Para aplicar una condición a cada elemento de un vector se usa `ifelse()`:

```
frecuencias <- c(45, 150, 890, 12, 340)

ifelse(frecuencias > 100, "alta", "baja")
#> [1] "baja" "alta" "alta" "baja" "alta"
```

Esta es la forma idiomática de crear variables categóricas a partir de numéricas, y la usarás constantemente con data frames:

```
corpus_meta$tamaño <- ifelse(corpus_meta$tokens > 2000, "grande", "pequeño")
```

Para más de dos categorías, `cut()` es más apropiada:

```
cut(frecuencias,
    breaks = c(0, 100, 500, Inf),
    labels = c("baja", "media", "alta"))
#> [1] baja  media alta  baja  media
#> Levels: baja media alta
```

Nota que `cut()` devuelve directamente un **factor**, con los levels ya ordenados.

3.6.3 Bucles: `for`

Un bucle `for` repite un bloque para cada elemento de un vector:

```
palabras <- c("casa", "lingüística", "sol")

for (p in palabras) {
  cat(p, "tiene", nchar(p), "caracteres\n")
}
#> casa tiene 4 caracteres
#> lingüística tiene 11 caracteres
#> sol tiene 3 caracteres
```

La variable `p` toma sucesivamente cada valor del vector. `cat()` imprime concatenando; `\n` es el salto de línea.

Preasignar el resultado

Cuando el bucle produce resultados que hay que guardar, la práctica correcta es **crear el contenedor antes**:

```
palabras <- c("casa", "lingüística", "sol", "árbol")

# Contenedor vacío del tamaño correcto
longitudes <- numeric(length(palabras))

for (i in seq_along(palabras)) {
  longitudes[i] <- nchar(palabras[i])
}

longitudes
#> [1]  4 11  3  5
```

Dos detalles:

- `seq_along(x)` genera `1:length(x)`, pero es seguro si el vector está vacío. Usarlo es buena práctica.
- Hacer crecer un objeto dentro del bucle (`resultado <- c(resultado, nuevo)`) funciona, pero es lento con muchos elementos porque R reserva memoria en cada iteración.

Cuándo NO usar un bucle

El ejemplo anterior no necesita bucle: `nchar()` está vectorizada.

```
nchar(palabras)
#> [1]  4 11  3  5
```

Regla práctica: si existe una versión vectorizada, úsala. Es más corta, más rápida y más legible. Reserva los bucles para operaciones que genuinamente dependen del paso anterior, o para procesos con efectos secundarios como leer varios archivos.

3.6.4 `while`

`while` repite mientras la condición sea verdadera. Se usa cuando no se sabe de antemano cuántas iteraciones harán falta:

```
tokens_acumulados <- 0
n_textos <- 0
tamaños <- c(1200, 3400, 890, 2100, 4200)

while (tokens_acumulados < 5000) {
  n_textos <- n_textos + 1
  tokens_acumulados <- tokens_acumulados + tamaños[n_textos]
}

cat("Se necesitan", n_textos, "textos para superar 5000 tokens\n")
#> Se necesitan 2 textos para superar 5000 tokens
```

Advertencia: si la condición nunca se vuelve falsa, el bucle es infinito. Verifica siempre que algo dentro del cuerpo modifique la variable de la condición.

3.6.5 Un caso completo: búsqueda en corpus

Combinemos lo anterior en algo que se parece a trabajo real. Simulamos un corpus de oraciones y buscamos un patrón:

```
corpus <- c(
  "El gato duerme en el sofá",
  "La casa grande está vacía",
  "El perro corre por el parque",
  "Los gatos son independientes",
  "Una casa pequeña junto al río"
)

# ¿Qué oraciones contienen "casa"?
grepl("casa", corpus)
#> [1] FALSE  TRUE FALSE FALSE  TRUE

# Extraerlas
corpus[grepl("casa", corpus)]
#> [1] "La casa grande está vacía"     "Una casa pequeña junto al río"

# Sus posiciones
```

```r
grep("casa", corpus)
#> [1] 2 5
```

`grepl()` devuelve lógicos; `grep()` devuelve índices. Ambas aceptan expresiones regulares, que son el instrumento central de la lingüística de corpus:

```r
# Palabras que empiezan por "gat"
grep("\\bgat", corpus, value = TRUE)
#> [1] "El gato duerme en el sofá"   "Los gatos son independientes"

# Ignorar mayúsculas
grep("^el", corpus, ignore.case = TRUE, value = TRUE)
#> [1] "El gato duerme en el sofá"   "El perro corre por el parque"
```

El argument `value = TRUE` hace que `grep()` devuelva los elementos en lugar de sus índices.

Ahora una función que encapsula la búsqueda y devuelve un resumen:

```r
buscar_patron <- function(corpus, patron) {
  coincidencias <- grep(patron, corpus, value = TRUE, ignore.case = TRUE)

  list(
    patron       = patron,
    n_encontrado = length(coincidencias),
    proporcion   = round(length(coincidencias) / length(corpus), 3),
    ejemplos     = coincidencias
  )
}

resultado <- buscar_patron(corpus, "casa")
resultado$n_encontrado
#> [1] 2
resultado$proporcion
#> [1] 0.4
```

Y finalmente, aplicando la búsqueda a varios patrones con `sapply()`:

```r
patrones <- c("casa", "gat", "perro")

sapply(patrones, function(p) length(grep(p, corpus, ignore.case = TRUE)))
#>  casa   gat perro
#>     2     2     1
```

3.6.6 Buenas prácticas

Cinco hábitos que conviene adquirir desde ahora:

1. Comenta el porqué, no el qué.

```r
# Mal: el código ya dice esto
x <- x + 1   # sumar 1 a x

# Bien: explica la decisión
# Se normaliza por millón porque el corpus de referencia usa esa base
freq_norm <- (freq / n_tokens) * 1000000
```

2. Nombres descriptivos. `frecuencias_normalizadas` es mejor que `fn`, y mucho mejor que `x2`. El código se escribe una vez y se lee muchas.

3. Fija la semilla cuando haya azar. Sin esto, tu análisis no es reproducible:

```r
set.seed(42)
muestra <- sample(1:1000, 50)
```

4. Un script, un propósito. Separa la carga de datos, el procesamiento y el análisis en archivos distintos, o al menos en secciones claramente marcadas. En RStudio, `# Sección ----` crea secciones plegables.

5. Verifica los supuestos de tus datos. Antes de analizar, comprueba:

```r
sum(is.na(datos$tokens))      # ¿hay valores faltantes?
any(datos$tokens < 0)         # ¿hay valores imposibles?
length(unique(datos$texto))   # ¿hay identificadores duplicados?
```

3.6.7 Sobre los errores

Vas a encontrar errores constantemente. Los tres más frecuentes al empezar:

```r
# 1. Objeto inexistente (típicamente, un typo)
frecuenciaa
#> Error: object 'frecuenciaa' not found

# 2. Paréntesis sin cerrar: R muestra + esperando el cierre
mean(c(1, 2, 3)
+

# 3. Función mal escrita
Mean(c(1,2,3))
#> Error in Mean(c(1, 2, 3)) : could not find function "Mean"
```

Leer el mensaje de error completo resuelve la mayoría de los casos. Si no, copiar el mensaje literal en un buscador casi siempre lleva a una respuesta en Stack Overflow —es una práctica normal y legítima, no un atajo.

Actividades 3.6

A6.1 Escribe un condicional que, dado un TTR, imprima `"alta diversidad léxica"` si supera 0.5, `"diversidad media"` si está entre 0.3 y 0.5, y `"baja diversidad"` si es menor. Pruébalo con tres valores.

A6.2 Convierte el condicional anterior en una función `evaluar_ttr()`. Aplícala al vector `c(0.62, 0.41, 0.22)` usando `sapply()`.

A6.3 Dado `frecuencias <- c(5, 120, 45, 890, 12, 340)`, crea con `ifelse()` un vector que clasifique cada valor como `"frecuente"` (> 100) o `"infrecuente"`.

A6.4 Resuelve la misma clasificación con `cut()`, pero con tres niveles: baja (0–50), media (51–300) y alta (más de 300). Verifica que el resultado es un factor y consulta sus levels.

A6.5 Escribe un bucle `for` que recorra un vector de cinco palabras e imprima, para cada una, la palabra en mayúsculas seguida de su número de caracteres. Después escribe la versión vectorizada equivalente y compara.

A6.6 Preasignando el contenedor, escribe un bucle que calcule el cuadrado de cada elemento de `1:10` y lo guarde en un vector.

A6.7 Usando el vector `corpus` de la sección 3.6.5, escribe expresiones que devuelvan: (a) las oraciones que contienen la palabra `"el"` como palabra completa, (b) las que empiezan con mayúscula seguida de vocal, (c) el número de oraciones que contienen alguna palabra terminada en `-s`.

A6.8 Escribe una función `perfil_corpus()` que reciba un vector de oraciones y devuelva una lista con: número de oraciones, número total de tokens (asumiendo separación por espacios), longitud media en tokens por oración y la oración más larga.

A6.9 Escribe un bucle `while` que sume elementos de un vector de frecuencias hasta superar el 50% del total, y devuelva cuántos elementos hicieron falta. (Este es el cálculo que subyace al concepto de *cobertura léxica*.)

A6.10 Revisa cualquiera de tus scripts anteriores y aplícale las cinco buenas prácticas de 3.6.6: comentarios que expliquen decisiones, nombres descriptivos, semilla fijada si hay azar, secciones marcadas y verificaciones de los datos.

Cierre

Checklist de autoevaluación

Antes de la primera sesión deberías poder responder que sí a todo lo siguiente. Si algo queda en no, vuelve a la sección correspondiente.

- ☐ Puedo asignar un valor a un objeto y consultar su `class`
- ☐ Sé consultar la ayuda de una función y localizar sus arguments
- ☐ Puedo escribir una función propia con un argument con valor por defecto
- ☐ Creo vectores, los indexo por posición, por nombre y por condición lógica
- ☐ Entiendo qué es la coerción y por qué `c(1, "a")` produce texto
- ☐ Distingo `sort()` de `order()`
- ☐ Sé qué es un factor, cómo reordenar sus levels y cómo declararlo ordinal
- ☐ Recuerdo que para extraer números de un factor se usa `as.numeric(as.character(x))`
- ☐ Creo data frames, los inspecciono con `str()` y filtro filas por condición
- ☐ Añado columnas calculadas y agrego por grupos con `aggregate()`
- ☐ Guardo y cargo un CSV cuidando la codificación
- ☐ Distingo `[ ]` de `[[ ]]` en listas y sé extraer el p-value de un test
- ☐ Escribo un condicional, conozco `ifelse()` y sé cuándo evitar un bucle
- ☐ Uso `grep()` y `grepl()` para buscar patrones en un vector de texto

Cómo entregar las actividades

Reúne las soluciones en **un solo script** de R (`.R`) o, si prefieres, en un documento R Markdown (`.Rmd`). Márcalo con secciones:

```
# ============================================
# Prelectura - Fundamentos de R
# Nombre: [tu nombre]
# ============================================

# --- Sección 3.1 ----------------------------

# A1.1
...
```

No hace falta que todo esté resuelto ni que todo funcione. Lo que interesa es que llegues habiendo intentado, con las dudas identificadas. Trae anotado lo que no salió: la primera sesión abre resolviéndolo.

Referencias

Fuente principal

- Gries, S. Th. (2009). *Quantitative Corpus Linguistics with R: A Practical Introduction*. Londres: Routledge. [Cap. 3]

Complementarias

- Gries, S. Th. (2021). *Statistics for Linguistics with R: A Practical Introduction* (3.ª ed.). Berlín: Mouton de Gruyter.
- Wickham, H. y Grolemund, G. (2017). *R for Data Science*. Sebastopol: O'Reilly. Disponible en https://r4ds.hadley.nz

- Levshina, N. (2015). *How to Do Linguistics with R*. Ámsterdam: John Benjamins.

Recursos de consulta rápida

- Cheatsheets de RStudio: https://posit.co/resources/cheatsheets/
- Documentación de expresiones regulares en R: `?regex`